

Java Interview Series · Part 5 · Q205–Q254

Top Java Coding Questions · Round 1

The 50 coding problems asked most in Java interviews, each with the optimal approach, time and space complexity, edge cases, and code that was run and verified on a real JDK.

50	50	15	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `TC-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

50	6	50	15
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🔢	01	Number Programs	Q205–Q212	5
🔤	02	String Programs	Q213–Q222	11
🧮	03	Array Programs	Q223–Q235	19
📶	04	Sorting & Selection	Q236–Q239	29
🔗	05	Linked Lists, Stacks & Queues	Q240–Q247	33
🌳	06	Trees, Graphs & Dynamic Programming	Q248–Q254	40

© TechTribers. For personal use only. Do not redistribute.

All questions

🔢 Number Programs

Q205 Swap two numbers without a temporary variable. 5

Q206 Check if a number is prime, and print all primes up to N. 6

Q207 Find the factorial of a number (iterative and recursive). How do you handle large values? 7

Q208 Print the Fibonacci series and find the n-th Fibonacci number efficiently. 7

Q209 Find the GCD and LCM of two numbers. 8

Q210 Check whether a number is an Armstrong number. 9

Q211 Reverse an integer and find the sum of its digits. How do you detect overflow? 10

Q212 Find the prime factors of a number. 10

🔤 String Programs

Q213 Reverse the words in a sentence (and reverse each word). 11

Q214 Find the first non-repeating character in a string. 12

Q215 Count the occurrences of each character and find the duplicate characters. 13

Q216 Find the length of the longest substring without repeating characters. 13

Q217 Check if a string of brackets is valid (balanced parentheses). 14

Q218 Find the longest common prefix of an array of strings. 15

Q219 Compress a string using counts of repeated characters ("aaabbc" → "a3b2c1"). 16

Q220 Check if one string is a rotation of another. 16

Q221 Find the longest palindromic substring. 17

Q222 Print all permutations of a string. 18

🧮 Array Programs

Q223 Find the second largest element in an array (in one pass). 19

Q224 Remove duplicates from a sorted array in place. 20

Q225 Two Sum: find two indices whose values add up to a target. 21

Q226 Move all zeros to the end while keeping the order of the other elements. 21

Q227 Find the missing number in an array containing 0…n (or 1…n). 22

Q228 Find the maximum subarray sum (Kadane's algorithm). 22

Q229 Rotate an array to the right by k steps in place. 23

Q230 Merge two sorted arrays into one sorted array. 24

Q231 Implement binary search. What is the classic bug? 25

Q232 Find duplicate elements in an array. 26

Q233 Best time to buy and sell a stock (one transaction). 26

Q234 Merge overlapping intervals. 27

Q235 Product of array except self (without division). 28

📶 Sorting & Selection

Q236 Implement bubble sort. Why is it rarely used? 29

Q237 Implement merge sort. 30

Q238 Implement quick sort. What is its worst case and how do you avoid it? 31

Q239 Find the k-th largest element in an array. 32

🔗 Linked Lists, Stacks & Queues

Q240 Reverse a singly linked list (iteratively and recursively). 33

Q241 Detect a cycle in a linked list (Floyd's algorithm). 34

Q242 Find the middle of a linked list in one pass. 35

Q243 Merge two sorted linked lists. 36

Q244 Remove the N-th node from the end of a linked list in one pass. 36

Q245 Design a stack that returns the minimum element in O(1). 37

Q246 Implement a queue using two stacks. 38

Q247 Implement an LRU cache with O(1) get and put. 39

© TechTribers. For personal use only. Do not redistribute.

🌳 Trees, Graphs & Dynamic Programming

Q248	Implement a binary search tree with insert, search and in-order traversal.	40
Q249	Find the height of a binary tree and print it level by level.	41
Q250	Check whether a binary tree is a valid binary search tree.	42
Q251	Find the lowest common ancestor (LCA) of two nodes in a binary tree.	43
Q252	Count the number of islands in a 2D grid.	44
Q253	Coin change: find the minimum number of coins to make an amount.	45
Q254	Find the longest common subsequence (LCS) of two strings.	45

© TechTribers. For personal use only. Do not redistribute.

Section 01

Number Programs

Warm-up questions from screening rounds. Interviewers watch for overflow, edge cases and complexity, not just a working loop.

Questions 205–212 · 8 questions



Q205 TC-001 • Basic

Swap two numbers without a temporary variable.

⚡ Quick answer

Use arithmetic (`a = a + b; b = a - b; a = a - b`) or XOR (`a ^= b; b ^= a; a ^= b`). Both are O(1); XOR can't overflow, while the sum trick can overflow but still works thanks to two's-complement wrap-around.

Java

```
int a = 10, b = 20;
a = a + b; b = a - b; a = a - b;
System.out.println("arithmetic: a=" + a + " b=" + b);

int x = 7, y = 3;
x ^= y; y ^= x; x ^= y;
System.out.println("xor: x=" + x + " y=" + y);

int big = Integer.MAX_VALUE, one = 1;
big = big + one; one = big - one; big = big - one;    // overflows mid-way, still correct
System.out.println("overflow case: " + big + " " + one);
```

▶ Output

```
arithmetic: a=20 b=10
xor: x=3 y=7
overflow case: 1 2147483647
```

⚠ Interview trap

XOR-swapping a variable with itself (`a ^= a`) zeroes it, which happens with array elements when `i == j` : `arr[i] ^= arr[j]` wipes the value. In real code, just use a temp variable: it's clearer and the JIT makes it free.

© TechTribers. For personal use only. Do not redistribute.

Q206 TC-002

• Basic

Check if a number is prime, and print all primes up to N.

⚡ Quick answer

For one number, trial-divide by 2 and odd numbers up to √n: O(√n). For all primes ≤ N, use the Sieve of Eratosthenes: O(N log log N) time, O(N) space.

Java

```java
static boolean isPrime(int n) {
    if (n < 2) return false;
    if (n % 2 == 0) return n == 2;
    for (int i = 3; (long) i * i <= n; i += 2)     // i*i avoids Math.sqrt and float issues
        if (n % i == 0) return false;
    return true;
}

static List<Integer> sieve(int n) {
    boolean[] composite = new boolean[n + 1];
    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= n; i++) {
        if (composite[i]) continue;
        primes.add(i);
        for (long j = (long) i * i; j <= n; j += i) composite[(int) j] = true;
    }
    return primes;
}

System.out.println(isPrime(97) + " " + isPrime(91) + " " + isPrime(2) + " " + isPrime(1));
System.out.println(sieve(30));
```

▶ Output

```
true false true false
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

⚠️ Interview trap

`i * i <= n` overflows for `n` near `Integer.MAX_VALUE` unless you compute it as `long`. Also, 1 is not prime and 2 is the only even prime.

© TechTribers. For personal use only. Do not redistribute.

! Q207 TC-003

• Basic

Find the factorial of a number (iterative and recursive). How do you handle large values?

⚡ Quick answer

Multiply 1…n in a loop (O(n), O(1) space) or recurse `n * fact(n-1)` (O(n) stack). `int` overflows after 12!, `long` after 20!; use `BigInteger` beyond that.

Java

```
static long factIter(int n) {
    if (n < 0) throw new IllegalArgumentException("n < 0");
    long f = 1;
    for (int i = 2; i <= n; i++) f = Math.multiplyExact(f, i);   // throws on overflow
    return f;
}
static long factRec(int n) { return n <= 1 ? 1 : n * factRec(n - 1); }
static BigInteger factBig(int n) {
    BigInteger f = BigInteger.ONE;
    for (int i = 2; i <= n; i++) f = f.multiply(BigInteger.valueOf(i));
    return f;
}

System.out.println(factIter(5) + " " + factRec(10) + " " + factIter(20));
System.out.println("25! = " + factBig(25));
try { factIter(21); } catch (ArithmeticException e) { System.out.println("21! overflows long"); }
```

▶ Output

```
120 3628800 2432902008176640000
25! = 15511210043330985984000000
21! overflows long
```

⚠️ Interview trap

Plain `int` / `long` multiplication overflows silently and returns garbage (13! as `int` is 1932053504). Use `Math.multiplyExact` or `BigInteger` and say so in the interview.

🌀 Q208 TC-004

• Basic

Print the Fibonacci series and find the n-th Fibonacci number efficiently.

⚡ Quick answer

Naive recursion is O(2^n) because it recomputes subproblems. Memoization (top-down) or iteration (bottom-up) makes it O(n); the iterative version needs only O(1) space.

© TechTribers. For personal use only. Do not redistribute.

```java
static long fibNaive(int n) { return n < 2 ? n : fibNaive(n - 1) + fibNaive(n - 2); }   // O(2^n)

static long fibMemo(int n, long[] memo) {
    if (n < 2) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
}

static long fibIter(int n) {
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long next = a + b; a = b; b = next; }
    return a;
}

System.out.println(IntStream.range(0, 10).mapToObj(i ->
String.valueOf(fibIter(i))).collect(Collectors.joining(" ")));
System.out.println(fibNaive(20) + " " + fibMemo(50, new long[51]) + " " + fibIter(90));
```

▶ Output

```
0 1 1 2 3 5 8 13 21 34
6765 12586269025 2880067194370816120
```

💡 **Pro tip**

This is the textbook example to explain **overlapping subproblems** and dynamic programming. For huge n (mod a prime), matrix exponentiation gives O(log n).

Q209 TC-005 • Basic

Find the GCD and LCM of two numbers.

⚡ **Quick answer**

Euclid's algorithm: `gcd(a, b) = gcd(b, a % b)` until `b == 0`, in O(log min(a, b)). Then `lcm(a, b) = a / gcd(a, b) * b` (divide first to avoid overflow).

```java
static long gcd(long a, long b) {
    a = Math.abs(a); b = Math.abs(b);
    while (b != 0) { long t = a % b; a = b; b = t; }
    return a;
}
static long lcm(long a, long b) { return a == 0 || b == 0 ? 0 : Math.abs(a / gcd(a, b) * b); }

System.out.println(gcd(48, 18) + " " + lcm(4, 6) + " " + gcd(17, 5) + " " + gcd(0, 9));
System.out.println(BigInteger.valueOf(48).gcd(BigInteger.valueOf(18)));        // JDK built-in
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
6 12 1 9
6
```

⚠️ **Interview trap**

`a * b / gcd(a, b)` overflows for large inputs even when the LCM itself fits. Always divide before multiplying.

💪 Q210 TC-006 • Basic

Check whether a number is an Armstrong number.

⚡ **Quick answer**

An Armstrong (narcissistic) number equals the sum of its digits each raised to the power of the digit count, like 153 = $1^3 + 5^3 + 3^3$. Count digits, then sum powers: O(d) for d digits.

Java

```
static boolean isArmstrong(int n) {
    if (n < 0) return false;
    int digits = String.valueOf(n).length();
    int sum = 0, tmp = n;
    while (tmp > 0) {
        int d = tmp % 10;
        int p = 1;
        for (int i = 0; i < digits; i++) p *= d;       // integer power, no Math.pow rounding
        sum += p;
        tmp /= 10;
    }
    return sum == n;
}
System.out.println(IntStream.range(1, 10_000).filter(Main::isArmstrong).boxed().toList());
```

▶ **Output**

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 153, 370, 371, 407, 1634, 8208, 9474]
```

⚠️ **Interview trap**

`(int) Math.pow(d, k)` returns a `double` and can round down on some values. Integer multiplication is exact and faster.

© TechTribers. For personal use only. Do not redistribute.

🔄 Q211 TC-007

• Intermediate

Reverse an integer and find the sum of its digits. How do you detect overflow?

⚡ Quick answer

Repeatedly take `n % 10` and `n /= 10`. When building the reversed number, check before `rev * 10 + d` that it won't exceed `Integer.MAX_VALUE` / `MIN_VALUE`, or use `Math.multiplyExact` / `addExact`.

Java

```
static int reverse(int n) {
    int rev = 0;
    while (n != 0) {
        int d = n % 10;                                  // negative for negative n: sign is kept
        if (rev > Integer.MAX_VALUE / 10 || rev < Integer.MIN_VALUE / 10) return 0;   // would
overflow
        rev = rev * 10 + d;
        n /= 10;
    }
    return rev;
}
static int digitSum(int n) {
    int sum = 0;
    for (n = Math.abs(n); n > 0; n /= 10) sum += n % 10;
    return sum;
}
System.out.println(reverse(12345) + " " + reverse(-120) + " " + reverse(1_534_236_469));
System.out.println(digitSum(98765) + " " + digitSum(-401));
```

▶ Output

```
54321 -21 0
35 5
```

🎯 Likely follow-up

Digital root (repeat digit-sum until one digit) has an O(1) formula: `n == 0 ? 0 : 1 + (n - 1) % 9`.

🧩 Q212 TC-008

• Basic

Find the prime factors of a number.

⚡ Quick answer

Divide out 2s, then try odd divisors `i` while `i * i ≤ n`, dividing out each fully. Whatever remains above 1 is itself prime. Time O(√n).

© TechTribers. For personal use only. Do not redistribute.

```java
static List<Long> primeFactors(long n) {
    List<Long> f = new ArrayList<>();
    while (n % 2 == 0) { f.add(2L); n /= 2; }
    for (long i = 3; i * i <= n; i += 2)
        while (n % i == 0) { f.add(i); n /= i; }
    if (n > 1) f.add(n);                                    // leftover prime
    return f;
}
System.out.println(primeFactors(360) + " " + primeFactors(97) + " " +
primeFactors(600_851_475_143L));
```

▶ Output

```
[2, 2, 2, 3, 3, 5] [97] [71, 839, 1471, 6857]
```

⚠️ **Interview trap**

The common version loops `i` all the way to `n`, which is O(n) and hangs on large primes. Stopping at √n and adding the leftover is the key insight.

Section 02

String Programs

The most frequently asked string problems, from character counting to sliding windows.

Questions 213–222 · 10 questions



Q213 TC-009 • Basic

Reverse the words in a sentence (and reverse each word).

⚡ **Quick answer**

Split on whitespace, reverse the word order, and join: O(n). To reverse each word in place instead, reverse characters inside each token.

```java
static String reverseWords(String s) {
    String[] words = s.trim().split("\\s+");                // handles multiple spaces
    Collections.reverse(Arrays.asList(words));
    return String.join(" ", words);
}
static String reverseEachWord(String s) {
```


© TechTribers. For personal use only. Do not redistribute.

```
    return Arrays.stream(s.trim().split("\\s+"))
            .map(w -> new StringBuilder(w).reverse().toString())
            .collect(Collectors.joining(" "));
}
System.out.println(reverseWords("  Java   is  awesome "));
System.out.println(reverseEachWord("Java is awesome"));
```

▶ Output

```
awesome is Java
avaJ si emosewa
```

⚠️ Interview trap

`s.split(" ")` on text with double spaces produces empty strings, so the output gets extra spaces. Use `trim()` plus the regex `\\s+`.

1️⃣ Q214 TC-010 • Basic

Find the first non-repeating character in a string.

⚡ Quick answer

Count frequencies in one pass, then scan the string again and return the first character with count 1. O(n) time; O(1) extra space for a fixed alphabet (an `int[26]` or `int[128]`).

Java

```
static Character firstUnique(String s) {
    int[] count = new int[128];                    // ASCII
    for (char c : s.toCharArray()) count[c]++;
    for (char c : s.toCharArray()) if (count[c] == 1) return c;
    return null;
}
static Optional<Character> firstUniqueStream(String s) {
    return s.chars().mapToObj(c -> (char) c)
            .collect(Collectors.groupingBy(Function.identity(), LinkedHashMap::new,
Collectors.counting()))
            .entrySet().stream().filter(e -> e.getValue() == 1).map(Map.Entry::getKey).findFirst();
}
System.out.println(firstUnique("swiss") + " " + firstUnique("techtribers") + " " +
firstUnique("aabb"));
System.out.println(firstUniqueStream("programming").orElse('-'));
```

▶ Output

```
w c null
p
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Interviewers often follow up: "now the string is a stream of characters". Keep a `LinkedHashMap` (or count array plus a queue) and evict characters as soon as they repeat.

📊 Q215 TC-011 • Basic

Count the occurrences of each character and find the duplicate characters.

> ⚡ **Quick answer**
> Use a `Map<Character, Long>` (`groupingBy` + `counting`) or an `int[]` count array, then filter counts greater than 1. O(n).

Java

```
String text = "programming";
Map<Character, Long> freq = text.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()));
System.out.println(freq);

List<Character> dups = freq.entrySet().stream()
        .filter(e -> e.getValue() > 1).map(Map.Entry::getKey).toList();
System.out.println("duplicates: " + dups);

// Remove duplicate characters, keeping first occurrence
StringBuilder unique = new StringBuilder();
Set<Character> seen = new HashSet<>();
for (char c : text.toCharArray()) if (seen.add(c)) unique.append(c);
System.out.println(unique);
```

▶ **Output**

```
{a=1, g=2, i=1, m=2, n=1, o=1, p=1, r=2}
duplicates: [g, m, r]
progamin
```

🪟 Q216 TC-012 • Intermediate

Find the length of the longest substring without repeating characters.

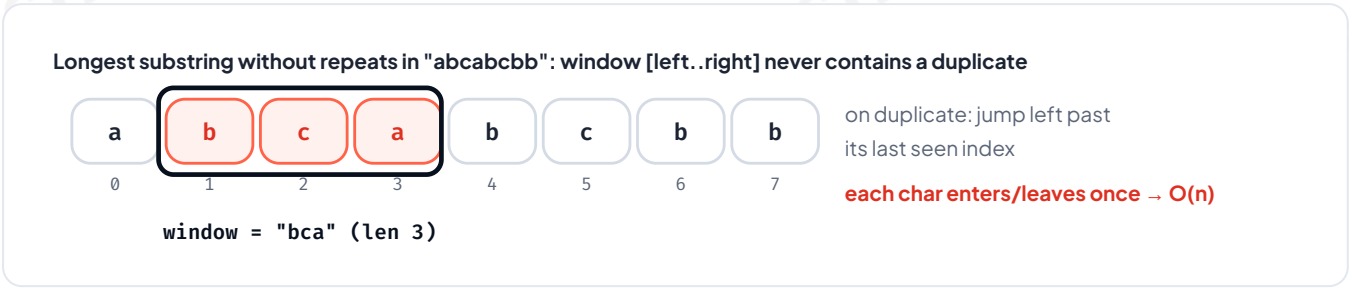
> ⚡ **Quick answer**
> Sliding window: expand `right`; when a character repeats inside the window, jump `left` past its previous index (stored in a map or array). Each index moves forward only, so it's O(n) time.

© TechTribers. For personal use only. Do not redistribute.

```java
static int longestUnique(String s) {
    int[] lastSeen = new int[128];
    Arrays.fill(lastSeen, -1);
    int best = 0, left = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastSeen[c] >= left) left = lastSeen[c] + 1;     // shrink past the duplicate
        lastSeen[c] = right;
        best = Math.max(best, right - left + 1);
    }
    return best;
}
for (String s : List.of("abcabcbb", "bbbbb", "pwwkew", "", "dvdf"))
    System.out.print(longestUnique(s) + " ");
```

▶ Output

```
3 1 3 0 3
```



💡 **Pro tip**

The sliding-window template (expand right, shrink left while invalid, record the best) solves a whole family: minimum window substring, at most K distinct characters, longest repeating replacement.

Q217 TC-013 • Basic

Check if a string of brackets is valid (balanced parentheses).

⚡ **Quick answer**

Push every opening bracket onto a stack; on a closing bracket, the stack top must be its matching opener. The string is valid if the stack is empty at the end. O(n) time and space.

```java
static boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        switch (c) {
            case '(' -> stack.push(')');
            case '[' -> stack.push(']');
```


© TechTribers. For personal use only. Do not redistribute.

```
            case '{' -> stack.push('}');
            default -> { if (stack.isEmpty() || stack.pop() != c) return false; }
        }
    }
    return stack.isEmpty();
}
for (String s : List.of("()[]{}", "{[()]}", "(]", "([)]", "((", ""))
    System.out.print(isValid(s) + " ");
```

▶ Output

```
true true false false false true
```

⚠️ **Interview trap**

Use `ArrayDeque` for stacks, not the legacy `java.util.Stack`, which extends `Vector` and synchronizes every call.

Q218 TC-014 • Basic

Find the longest common prefix of an array of strings.

⚡ **Quick answer**

Vertical scan: compare character `i` of every string and stop at the first mismatch or the shortest string's end. O(S) where S is the total number of characters.

Java

```
static String lcp(String... words) {
    if (words.length == 0) return "";
    for (int i = 0; i < words[0].length(); i++) {
        char c = words[0].charAt(i);
        for (String w : words)
            if (i == w.length() || w.charAt(i) != c) return words[0].substring(0, i);
    }
    return words[0];
}
System.out.println(lcp("flower", "flow", "flight") + "|" + lcp("dog", "racecar", "car") + "|" +
lcp("java", "javascript", "javac"));
```

▶ Output

```
fl||java
```

💡 **Pro tip**

Alternative: sort the array and compare only the first and last strings; their common prefix is everyone's. That costs O(n log n · L) for the sort, so the vertical scan is better.

© TechTribers. For personal use only. Do not redistribute.

Q219 TC-015

• Basic

Compress a string using counts of repeated characters ("aaabbc" → "a3b2c1").

⚡ Quick answer

Walk the string with a run counter; when the character changes, append the character and its count. Return the original if the compressed form isn't shorter. O(n).

Java

```
static String compress(String s) {
    if (s.isEmpty()) return s;
    StringBuilder sb = new StringBuilder();
    int run = 1;
    for (int i = 1; i <= s.length(); i++) {
        if (i < s.length() && s.charAt(i) == s.charAt(i - 1)) { run++; continue; }
        sb.append(s.charAt(i - 1)).append(run);
        run = 1;
    }
    return sb.length() < s.length() ? sb.toString() : s;
}
System.out.println(compress("aaabbc") + " " + compress("aabccccaaa") + " " + compress("abc"));
```

▶ Output

```
aaabbc a2b1c5a3 abc
```

⚠️ Interview trap

For "aaabbc" the compressed form `a3b2c1` is 6 characters, the same as the input, so the method returns the original. Clarify this rule with the interviewer before coding.

Q220 TC-016

• Basic

Check if one string is a rotation of another.

⚡ Quick answer

`s2` is a rotation of `s1` exactly when the lengths match and `s2` occurs inside `s1 + s1`. O(n) with a linear-time search; `contains()` is fine in interviews.

© TechTribers. For personal use only. Do not redistribute.

```java
static boolean isRotation(String s1, String s2) {
    return s1.length() == s2.length() && (s1 + s1).contains(s2);
}
System.out.println(isRotation("waterbottle", "erbottlewat") + " " + isRotation("abcd", "acbd") + " "
+ isRotation("aa", "a"));
```

▶ Output

```
true false false
```

⚠️ **Interview trap**

Forgetting the length check makes `"a"` look like a rotation of `"aa"`.

Q221 TC-017 • Intermediate

Find the longest palindromic substring.

⚡ **Quick answer**

Expand around each centre (n single-character centres plus n−1 between characters) while the ends match, and keep the longest. O(n²) time, O(1) space, simpler than the DP table. Manacher's algorithm achieves O(n).

```java
static String longestPalindrome(String s) {
    if (s.length() < 2) return s;
    int start = 0, len = 1;
    for (int c = 0; c < s.length(); c++) {
        for (int[] lr : new int[][]{{c, c}, {c, c + 1}}) {          // odd and even centres
            int l = lr[0], r = lr[1];
            while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; }
            if (r - l - 1 > len) { len = r - l - 1; start = l + 1; }
        }
    }
    return s.substring(start, start + len);
}
System.out.println(longestPalindrome("babad") + " " + longestPalindrome("cbbd") + " " +
longestPalindrome("forgeeksskeegfor"));
```

▶ Output

```
bab bb geeksskeeg
```


© TechTribers. For personal use only. Do not redistribute.

Q222 TC-018

• Intermediate

Print all permutations of a string.

⚡ Quick answer

Backtracking: fix each unused character at the current position, recurse on the rest, then undo. There are n! permutations, so it's O(n · n!) time. Skip a character equal to one already tried at the same position to avoid duplicates.

Java

```
static void permute(char[] chars, boolean[] used, StringBuilder cur, List<String> out) {
    if (cur.length() == chars.length) { out.add(cur.toString()); return; }
    for (int i = 0; i < chars.length; i++) {
        if (used[i] || (i > 0 && chars[i] == chars[i - 1] && !used[i - 1])) continue;   // skip
duplicate branches
        used[i] = true; cur.append(chars[i]);
        permute(chars, used, cur, out);
        cur.deleteCharAt(cur.length() - 1); used[i] = false;                          // backtrack
    }
}
static List<String> permutations(String s) {
    char[] c = s.toCharArray();
    Arrays.sort(c);                                  // sorting groups duplicates together
    List<String> out = new ArrayList<>();
    permute(c, new boolean[c.length], new StringBuilder(), out);
    return out;
}
System.out.println(permutations("abc"));
System.out.println(permutations("aab") + " count=" + permutations("java").size());
```

▶ Output

```
[abc, acb, bac, bca, cab, cba]
[aab, aba, baa] count=12
```

💡 Pro tip

Backtracking (choose, explore, un-choose) is the same template for subsets, combinations, N-Queens and Sudoku. Name the pattern in the interview.

© TechTribers. For personal use only. Do not redistribute.

Section 03

Array Programs

Two pointers, hashing, prefix tricks and Kadane: the array patterns behind most screening questions.

Questions 223–235 · 13 questions



Q223 TC-019

• Basic

Find the second largest element in an array (in one pass).

⚡ Quick answer

Track `largest` and `second` in a single scan, updating `second` only for values strictly between them. O(n) time, O(1) space, and it handles duplicates correctly.

Java

```
static OptionalInt secondLargest(int[] a) {
    long first = Long.MIN_VALUE, second = Long.MIN_VALUE;
    for (int x : a) {
        if (x > first) { second = first; first = x; }
        else if (x < first && x > second) second = x;          // strictly smaller than first
    }
    return second == Long.MIN_VALUE ? OptionalInt.empty() : OptionalInt.of((int) second);
}
System.out.println(secondLargest(new int[]{10, 25, 7, 42, 32}) + " "
        + secondLargest(new int[]{42, 42, 7}) + " " + secondLargest(new int[]{5, 5}));
```

▶ Output

```
OptionalInt[32] OptionalInt[7] OptionalInt.empty
```

⚠ Interview trap

The popular `Arrays.sort(a); return a[a.length - 2];` is O(n log n), mutates the input, and returns 42 for `{42, 42, 7}`. Duplicates are the first thing interviewers test.

© TechTribers. For personal use only. Do not redistribute.

Q224 TC-020

• Basic

Remove duplicates from a sorted array in place.

⚡ Quick answer

Two pointers: `write` marks the end of the unique prefix; for each element that differs from the last written one, copy it to `write++`. Returns the new length. O(n) time, O(1) space.

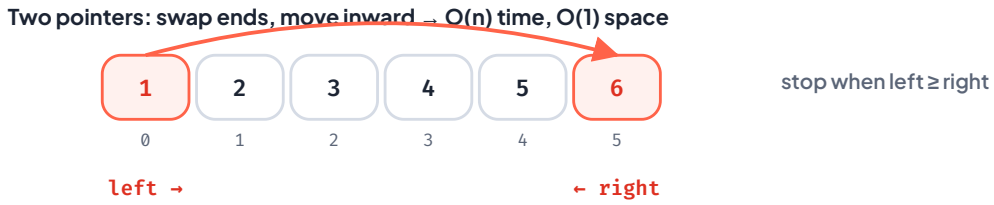
Java

```
static int dedupSorted(int[] a) {
    if (a.length == 0) return 0;
    int write = 1;
    for (int read = 1; read < a.length; read++)
        if (a[read] != a[write - 1]) a[write++] = a[read];
    return write;
}
int[] nums = {1, 1, 2, 3, 3, 3, 4, 5, 5};
int k = dedupSorted(nums);
System.out.println(k + " " + Arrays.toString(Arrays.copyOf(nums, k)));

int[] unsorted = {4, 1, 4, 2, 1};
System.out.println(Arrays.toString(Arrays.stream(unsorted).distinct().toArray()));   // unsorted: keep order
```

▶ Output

```
5 [1, 2, 3, 4, 5]
[4, 1, 2]
```



⚠️ Interview trap

The adjacent-comparison trick only works on **sorted** input. For unsorted arrays use a `LinkedHashSet` or `distinct()` (O(n) time, O(n) space).

© TechTribers. For personal use only. Do not redistribute.

🎯 Q225 TC-021 • Basic

Two Sum: find two indices whose values add up to a target.

⚡ **Quick answer**

One pass with a `HashMap<value, index>`: for each element, check whether `target - value` was already seen. O(n) time, O(n) space, versus O(n²) for nested loops.

Java

```
static int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        Integer j = seen.get(target - nums[i]);
        if (j != null) return new int[]{j, i};
        seen.put(nums[i], i);
    }
    return new int[0];
}
System.out.println(Arrays.toString(twoSum(new int[]{2, 7, 11, 15}, 9)) + " "
        + Arrays.toString(twoSum(new int[]{3, 3}, 6)) + " " + Arrays.toString(twoSum(new int[]{1,
2}, 7)));
```

▶ **Output**

```
[0, 1] [0, 1] []
```

🎯 **Likely follow-up**

If the array is sorted, use two pointers from both ends: O(n) time and O(1) space. That variant is the building block for 3Sum.

0️⃣ Q226 TC-022 • Basic

Move all zeros to the end while keeping the order of the other elements.

⚡ **Quick answer**

Copy every non-zero element forward to `write++`, then fill the rest with zeros. O(n) time, O(1) space, stable.

Java

```
static void moveZeros(int[] a) {
    int write = 0;
    for (int x : a) if (x != 0) a[write++] = x;
    while (write < a.length) a[write++] = 0;
}
int[] a = {0, 1, 0, 3, 12, 0, 7};
```


© TechTribers. For personal use only. Do not redistribute.

```
moveZeros(a);
System.out.println(Arrays.toString(a));
```

▶ Output

```
[1, 3, 12, 7, 0, 0, 0]
```

Q227 TC-023

• Basic

Find the missing number in an array containing 0…n (or 1…n).

⚡ Quick answer

The expected sum is n(n+1)/2; subtract the actual sum. Or XOR all indices and values: pairs cancel and the missing number remains, with no overflow risk. Both O(n) time, O(1) space.

Java

```
static int missingSum(int[] a) {            // values 0..n with one missing, length n
    long n = a.length;
    long expected = n * (n + 1) / 2;
    long actual = 0;
    for (int x : a) actual += x;
    return (int) (expected - actual);
}
static int missingXor(int[] a) {
    int x = a.length;
    for (int i = 0; i < a.length; i++) x ^= i ^ a[i];
    return x;
}
int[] nums = {3, 0, 1, 5, 2};
System.out.println(missingSum(nums) + " " + missingXor(nums));
```

▶ Output

```
4 4
```

⚠ Interview trap

Computing the sum in `int` overflows for large n. Use `long`, or the XOR version.

Q228 TC-024

• Intermediate

Find the maximum subarray sum (Kadane's algorithm).

⚡ Quick answer

Scan once keeping `current = max(x, current + x)` (best sum ending here) and `best = max(best, current)`. O(n) time, O(1) space. Start from the first element so all-negative arrays work.

© TechTribers. For personal use only. Do not redistribute.

Java

```java
static int[] maxSubarray(int[] a) {              // returns {sum, start, end}
    int best = a[0], cur = a[0], start = 0, bestStart = 0, bestEnd = 0;
    for (int i = 1; i < a.length; i++) {
        if (a[i] > cur + a[i]) { cur = a[i]; start = i; } else cur += a[i];
        if (cur > best) { best = cur; bestStart = start; bestEnd = i; }
    }
    return new int[]{best, bestStart, bestEnd};
}
int[] r = maxSubarray(new int[]{-2, 1, -3, 4, -1, 2, 1, -5, 4});
System.out.println("sum=" + r[0] + " from " + r[1] + " to " + r[2]);
System.out.println("all negative: " + maxSubarray(new int[]{-8, -3, -6})[0]);
```

▶ Output

```
sum=6 from 3 to 6
all negative: -3
```

Kadane: at each index, extend the running sum or restart from this element

nums	-2	1	-3	4	-1	2	1	-5	4
best ending	-2	1	-2	4	3	5	6	1	5

`max subarray = [4, -1, 2, 1] = 6 · O(n) time · O(1) space`

⚠️ **Interview trap**

Initialising `best = 0` returns 0 for an all-negative array, which is wrong unless empty subarrays are allowed. Ask, then initialise with the first element.

🔃 **Q229** TC-025 • Intermediate

Rotate an array to the right by k steps in place.

⚡ **Quick answer**

Reverse the whole array, then reverse the first `k` and the remaining `n - k` elements. O(n) time, O(1) space. Normalise `k %= n` first.

Java

```java
static void reverse(int[] a, int i, int j) { while (i < j) { int t = a[i]; a[i++] = a[j]; a[j--] =
t; } }
static void rotateRight(int[] a, int k) {
    int n = a.length;
    if (n == 0) return;
    k %= n;
```


© TechTribers. For personal use only. Do not redistribute.

```
    reverse(a, 0, n - 1);
    reverse(a, 0, k - 1);
    reverse(a, k, n - 1);
}
int[] a = {1, 2, 3, 4, 5, 6, 7};
rotateRight(a, 3);
System.out.println(Arrays.toString(a));
int[] b = {1, 2};
rotateRight(b, 5);                                  // k > n
System.out.println(Arrays.toString(b));
```

▶ Output

```
[5, 6, 7, 1, 2, 3, 4]
[2, 1]
```

💡 Pro tip

For lists, the JDK has it built in: `Collections.rotate(list, k)`.

Q230 TC-026 • Basic

Merge two sorted arrays into one sorted array.

⚡ Quick answer

Two pointers: repeatedly take the smaller head element, then copy whatever remains. O(m + n) time. To merge in place into the first array's spare capacity, fill from the back.

Java

```
static int[] merge(int[] a, int[] b) {
    int[] out = new int[a.length + b.length];
    int i = 0, j = 0, k = 0;
    while (i < a.length && j < b.length) out[k++] = a[i] <= b[j] ? a[i++] : b[j++];   // <= keeps it
stable
    while (i < a.length) out[k++] = a[i++];
    while (j < b.length) out[k++] = b[j++];
    return out;
}
System.out.println(Arrays.toString(merge(new int[]{1, 4, 7, 9}, new int[]{2, 3, 8, 10, 12})));
```

▶ Output

```
[1, 2, 3, 4, 7, 8, 9, 10, 12]
```


© TechTribers. For personal use only. Do not redistribute.

Q231 TC-027

• Basic

Implement binary search. What is the classic bug?

⚡ Quick answer

On a sorted array, compare with the middle element and discard half the range each step: O(log n). Compute `mid = lo + (hi - lo) / 2` (or `(lo + hi) >>> 1`) to avoid integer overflow.

Java

```
static int binarySearch(int[] a, int target) {
    int lo = 0, hi = a.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) lo = mid + 1; else hi = mid - 1;
    }
    return -(lo + 1);                                  // same contract as Arrays.binarySearch
}
int[] a = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
System.out.println(binarySearch(a, 23) + " " + binarySearch(a, 2) + " " + binarySearch(a, 50) + " "
+ Arrays.binarySearch(a, 50));
```

▶ Output

```
5 0 -8 -8
```

Binary search for 23: halve the range each step → O(log n)

step 1	2	5	8	12	**16**	23	38	56	72	91
step 2	2	5	8	12	16	23	38	**56**	72	91
step 3	2	5	8	12	16	**23**	38	56	72	91

`mid = lo + (hi - lo) / 2 (avoids int overflow of (lo + hi) / 2)`

⚠️ Interview trap

`(lo + hi) / 2` overflows when `lo + hi > Integer.MAX_VALUE`. This exact bug sat in the JDK's own `Arrays.binarySearch` for about nine years until it was fixed in 2006.

© TechTribers. For personal use only. Do not redistribute.

Q232 TC-028 • Basic

Find duplicate elements in an array.

⚡ Quick answer

Use a `HashSet` : `add()` returns `false` for an element already seen. O(n) time, O(n) space. If values are in 1…n, you can mark visited indexes by negating values for O(1) extra space.

Java

```
static Set<Integer> duplicates(int[] a) {
    Set<Integer> seen = new HashSet<>(), dups = new TreeSet<>();
    for (int x : a) if (!seen.add(x)) dups.add(x);
    return dups;
}
static List<Integer> duplicatesInPlace(int[] a) {           // values in 1..n
    List<Integer> out = new ArrayList<>();
    for (int i = 0; i < a.length; i++) {
        int idx = Math.abs(a[i]) - 1;
        if (a[idx] < 0) out.add(Math.abs(a[i])); else a[idx] = -a[idx];
    }
    return out;
}
System.out.println(duplicates(new int[]{4, 3, 2, 7, 8, 2, 3, 1}) + " " + duplicatesInPlace(new int[]
{4, 3, 2, 7, 8, 2, 3, 1}));
```

▶ Output

```
[2, 3] [2, 3]
```

⚠️ Interview trap

The in-place trick modifies the input array. Mention it, and restore values with `Math.abs` afterwards if the caller needs them.

Q233 TC-029 • Basic

Best time to buy and sell a stock (one transaction).

⚡ Quick answer

Track the minimum price so far and the best profit `price - minSoFar` in one pass. O(n) time, O(1) space.

Java

```
static int maxProfit(int[] prices) {
    int min = Integer.MAX_VALUE, best = 0;
    for (int p : prices) {
        min = Math.min(min, p);
        best = Math.max(best, p - min);
    }
```


© TechTribers. For personal use only. Do not redistribute.

```
    return best;
}
static int maxProfitUnlimited(int[] prices) {          // any number of transactions
    int profit = 0;
    for (int i = 1; i < prices.length; i++) profit += Math.max(0, prices[i] - prices[i - 1]);
    return profit;
}
int[] prices = {7, 1, 5, 3, 6, 4};
System.out.println(maxProfit(prices) + " " + maxProfitUnlimited(prices) + " " + maxProfit(new int[]
{7, 6, 4, 3}));
```

▶ Output

```
5 7 0
```

Q234 TC-030

• Intermediate

Merge overlapping intervals.

⚡ Quick answer

Sort intervals by start; then for each interval, extend the last merged interval if it overlaps (`start ≤ last.end`), otherwise start a new one. O(n log n) for the sort.

Java

```
static List<int[]> mergeIntervals(int[][] in) {
    int[][] iv = in.clone();
    Arrays.sort(iv, Comparator.comparingInt(x -> x[0]));
    List<int[]> out = new ArrayList<>();
    for (int[] cur : iv) {
        if (out.isEmpty() || out.get(out.size() - 1)[1] < cur[0]) out.add(cur.clone());
        else out.get(out.size() - 1)[1] = Math.max(out.get(out.size() - 1)[1], cur[1]);
    }
    return out;
}
List<int[]> m = mergeIntervals(new int[][]{{8, 10}, {1, 3}, {2, 6}, {15, 18}, {17, 20}});
System.out.println(m.stream().map(Arrays::toString).collect(Collectors.joining(" ")));
```

▶ Output

```
[1, 6] [8, 10] [15, 20]
```



sort by start, then extend the last merged interval while next.start <= last.end -> O(n log n)

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Sorting with `(a, b) -> a[0] - b[0]` (seen in many solutions) overflows for large negative and positive values. Use `Comparator.comparingInt`.

✖ Q235 TC-031 • Intermediate

Product of array except self (without division).

⚡ **Quick answer**

Build prefix products left-to-right into the output array, then multiply by suffix products right-to-left using a running variable. O(n) time, O(1) extra space besides the output.

Java

```
static int[] productExceptSelf(int[] a) {
    int n = a.length;
    int[] out = new int[n];
    out[0] = 1;
    for (int i = 1; i < n; i++) out[i] = out[i - 1] * a[i - 1];       // product of everything left
    int right = 1;
    for (int i = n - 1; i >= 0; i--) { out[i] *= right; right *= a[i]; }   // times everything right
    return out;
}
System.out.println(Arrays.toString(productExceptSelf(new int[]{1, 2, 3, 4})) + " "
        + Arrays.toString(productExceptSelf(new int[]{-1, 1, 0, -3, 3})));
```

▶ **Output**

```
[24, 12, 8, 6] [0, 0, 9, 0, 0]
```

⚠️ **Interview trap**

Total product divided by each element fails on zeros, which is exactly why interviewers forbid division.

© TechTribers. For personal use only. Do not redistribute.

Section 04

Sorting & Selection

Implement the classics, know their complexity and stability, and pick the right tool for top-K problems.

Questions 236–239 · 4 questions



Q236 TC-032

• Basic

Implement bubble sort. Why is it rarely used?

⚡ Quick answer

Repeatedly swap adjacent out-of-order pairs; after pass i the largest i elements are in place. Add an early exit when a pass makes no swaps. O(n²) average and worst, O(n) best (already sorted), stable, in place.

Algorithm	Best	Average	Worst	Space	Stable
Bubble / Insertion	O(n)	O(n²)	O(n²)	O(1)	Yes
Selection	O(n²)	O(n²)	O(n²)	O(1)	No
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick	O(n log n)	O(n log n)	O(n²)	O(log n)	No
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No

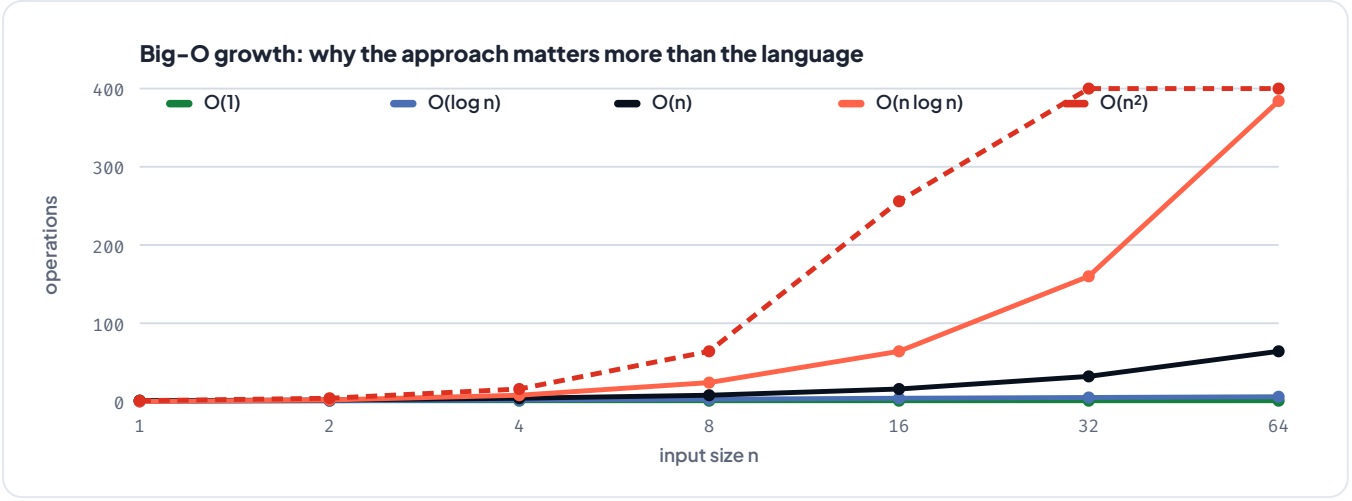
Java

```java
static void bubbleSort(int[] a) {
    for (int pass = 0; pass < a.length - 1; pass++) {
        boolean swapped = false;
        for (int j = 0; j < a.length - 1 - pass; j++) {
            if (a[j] > a[j + 1]) { int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t; swapped = true; }
        }
        if (!swapped) break;                          // already sorted
    }
}
int[] a = {64, 34, 25, 12, 22, 11, 90};
bubbleSort(a);
System.out.println(Arrays.toString(a));
```

▶ Output

```
[11, 12, 22, 25, 34, 64, 90]
```


© TechTribers. For personal use only. Do not redistribute.



💡 Pro tip

What the JDK actually uses: `Arrays.sort(int[])` is dual-pivot quicksort; `Arrays.sort(Object[])` and `List.sort` use TimSort (stable, adaptive merge sort), which is O(n) on already-sorted data.

Q237 TC-033 • Intermediate

Implement merge sort.

⚡ Quick answer

Split the array in half, sort each half recursively, then merge the two sorted halves. O(n log n) in all cases, stable, O(n) extra space, and the go-to for linked lists and external sorting.

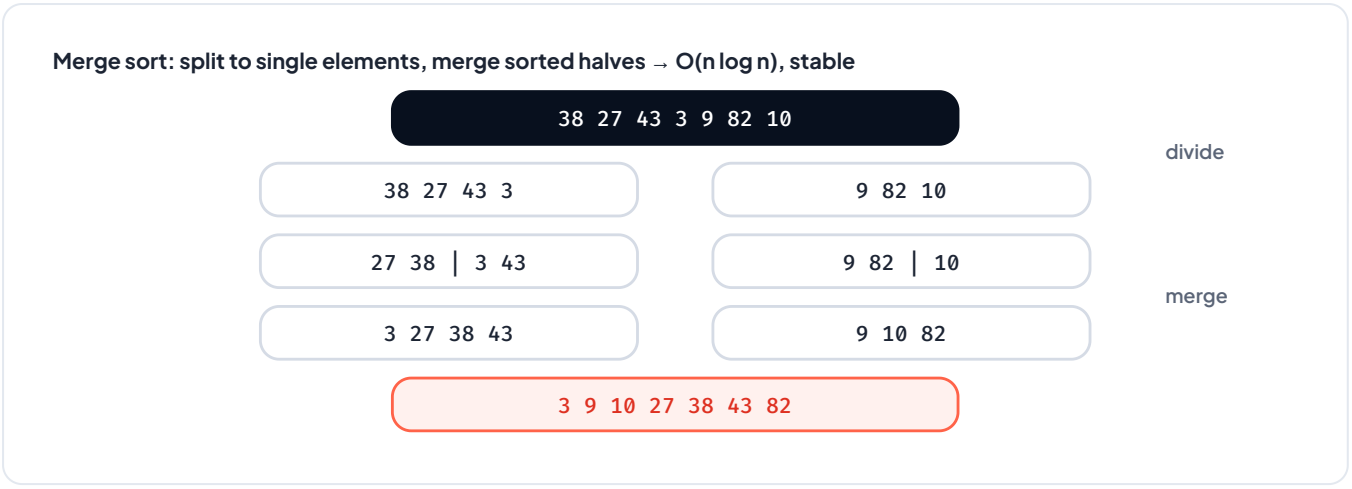
Java

```java
static void mergeSort(int[] a, int[] tmp, int lo, int hi) {     // sorts a[lo..hi)
    if (hi - lo < 2) return;
    int mid = (lo + hi) >>> 1;
    mergeSort(a, tmp, lo, mid);
    mergeSort(a, tmp, mid, hi);
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi) tmp[k++] = a[i] <= a[j] ? a[i++] : a[j++];
    while (i < mid) tmp[k++] = a[i++];
    while (j < hi) tmp[k++] = a[j++];
    System.arraycopy(tmp, lo, a, lo, hi - lo);
}
int[] a = {38, 27, 43, 3, 9, 82, 10};
mergeSort(a, new int[a.length], 0, a.length);
System.out.println(Arrays.toString(a));
```

▶ Output

```
[3, 9, 10, 27, 38, 43, 82]
```


© TechTribers. For personal use only. Do not redistribute.



💡 Pro tip

Allocate the temp buffer once and pass it down. Allocating a new array in every recursive call is the most common performance mistake in interview solutions.

⚡ Q238 TC-034 • Intermediate

Implement quick sort. What is its worst case and how do you avoid it?

⚡ Quick answer

Pick a pivot, partition so smaller elements go left and larger go right, then recurse on both sides. Average O(n log n), in place; worst case O(n²) on bad pivots (e.g. sorted input with first/last pivot), avoided with a random or median-of-three pivot.

Java

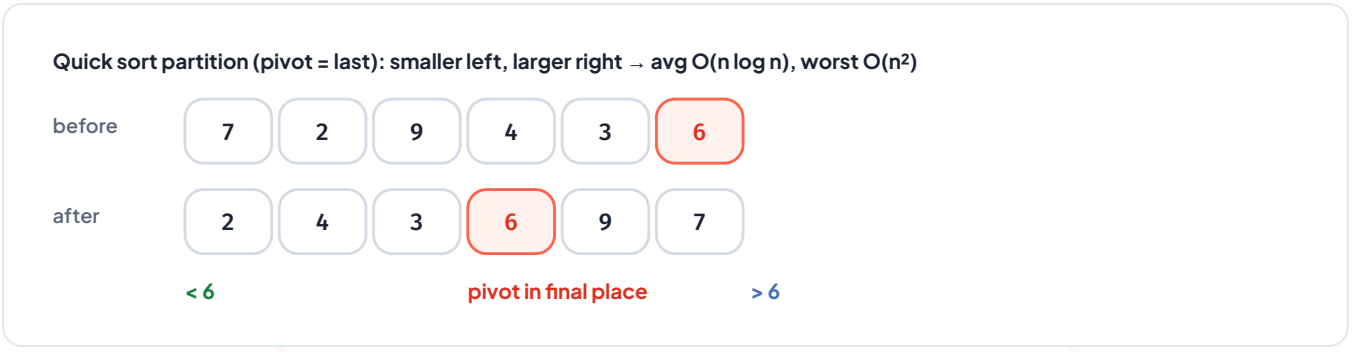
```java
static void quickSort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int r = lo + ThreadLocalRandom.current().nextInt(hi - lo + 1);   // random pivot
    swap(a, r, hi);
    int pivot = a[hi], i = lo;
    for (int j = lo; j < hi; j++) if (a[j] < pivot) swap(a, i++, j);   // Lomuto partition
    swap(a, i, hi);
    quickSort(a, lo, i - 1);
    quickSort(a, i + 1, hi);
}
static void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }

int[] a = {7, 2, 9, 4, 3, 6, 1, 8, 5};
quickSort(a, 0, a.length - 1);
System.out.println(Arrays.toString(a));
```

▶ Output

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```


© TechTribers. For personal use only. Do not redistribute.



🎯 Likely follow-up

Quick sort vs merge sort? Quick sort is usually faster in memory (in place, cache-friendly) but not stable; merge sort guarantees O(n log n) and is stable, which is why Java uses a merge-sort variant for objects.

🏆 **Q239** TC-035 • Intermediate

Find the k-th largest element in an array.

⚡ Quick answer

Keep a min-heap of size k: push each element and pop when size exceeds k; the heap top is the answer. O(n log k) time, O(k) space. Quickselect gives O(n) on average.

Java

```
static int kthLargest(int[] a, int k) {
    PriorityQueue<Integer> heap = new PriorityQueue<>();          // min-heap
    for (int x : a) {
        heap.offer(x);
        if (heap.size() > k) heap.poll();                          // drop the smallest
    }
    return heap.peek();
}
static List<String> topKFrequent(List<String> words, int k) {
    Map<String, Long> freq = words.stream().collect(Collectors.groupingBy(w -> w,
Collectors.counting()));
    return freq.entrySet().stream()
            .sorted(Map.Entry.<String,
Long>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey()))
            .limit(k).map(Map.Entry::getKey).toList();
}
System.out.println(kthLargest(new int[]{3, 2, 1, 5, 6, 4}, 2) + " " + kthLargest(new int[]{3, 2, 3,
1, 2, 4, 5, 5, 6}, 4));
System.out.println(topKFrequent(List.of("java", "kafka", "java", "spring", "kafka", "java"), 2));
```

▶ Output

```
5 4
[java, kafka]
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

For "k-th smallest", flip it: use a max-heap, `new PriorityQueue<>(Comparator.reverseOrder())`.

Section 05

Linked Lists, Stacks & Queues

Pointer manipulation and the classic data-structure design questions, including the LRU cache.

Questions 240–247 · 8 questions



Q240 TC-036 • Basic

Reverse a singly linked list (iteratively and recursively).

⚡ **Quick answer**

Iteratively: walk the list re-pointing each `next` to the previous node, using three pointers (`prev`, `cur`, `next`). O(n) time, O(1) space. Recursion is elegant but uses O(n) stack.

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }

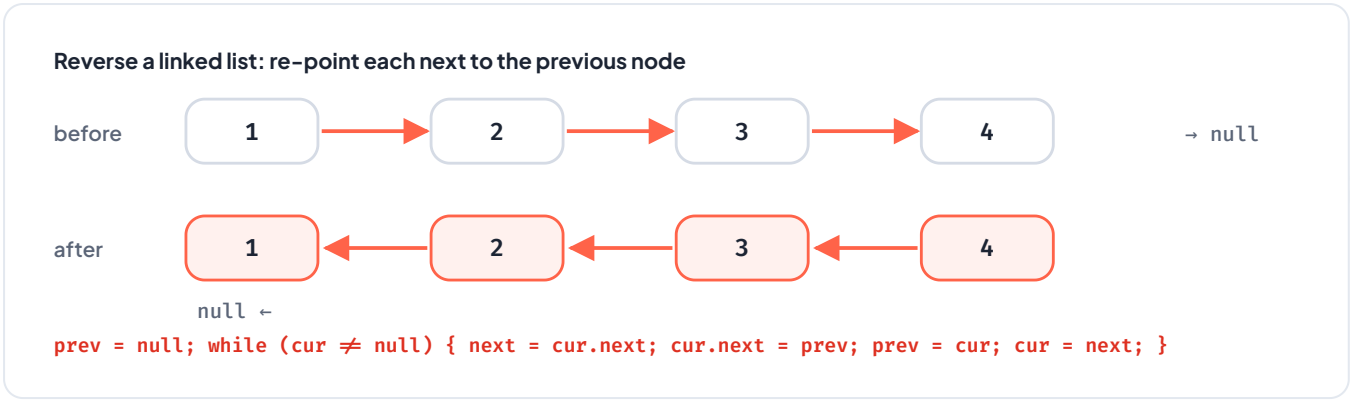
static Node reverse(Node head) {
    Node prev = null, cur = head;
    while (cur != null) { Node next = cur.next; cur.next = prev; prev = cur; cur = next; }
    return prev;
}
static Node reverseRec(Node head) {
    if (head == null || head.next == null) return head;
    Node newHead = reverseRec(head.next);
    head.next.next = head;
    head.next = null;
    return newHead;
}
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i],
h); return h; }
static String str(Node h) { StringBuilder sb = new StringBuilder(); for (; h != null; h = h.next)
sb.append(h.val).append(h.next != null ? "->" : ""); return sb.toString(); }

System.out.println(str(reverse(of(1, 2, 3, 4, 5))) + " | " + str(reverseRec(of(1, 2, 3))));
```

▶ **Output**

```
5->4->3->2->1 | 3->2->1
```


© TechTribers. For personal use only. Do not redistribute.



Q241 TC-037

• Intermediate

Detect a cycle in a linked list (Floyd's algorithm).

⚡ **Quick answer**

Move `slow` one step and `fast` two steps. If they ever meet there is a cycle; if `fast` reaches `null` there isn't. O(n) time, O(1) space. To find the cycle start, reset one pointer to `head` and move both one step until they meet.

Java

```
static final class Node { int val; Node next; Node(int v) { val = v; } }

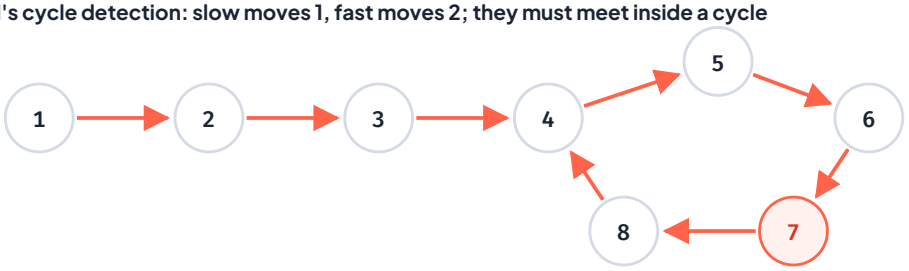
static Node cycleStart(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
        slow = slow.next; fast = fast.next.next;
        if (slow == fast) {                                  // cycle found
            for (slow = head; slow != fast; slow = slow.next, fast = fast.next) { }
            return slow;
        }
    }
    return null;
}
Node[] n = new Node[8];
for (int i = 0; i < 8; i++) n[i] = new Node(i + 1);
for (int i = 0; i < 7; i++) n[i].next = n[i + 1];
n[7].next = n[3];                                        // 8 -> 4 creates the cycle
System.out.println("cycle starts at " + cycleStart(n[0]).val);
n[7].next = null;
System.out.println("after fix: " + cycleStart(n[0]));
```

▶ **Output**

```
cycle starts at 4
after fix: null
```


© TechTribers. For personal use only. Do not redistribute.

Floyd's cycle detection: slow moves 1, fast moves 2; they must meet inside a cycle



⚠️ **Interview trap**

A `HashSet<Node>` of visited nodes also works, but uses O(n) memory. Interviewers asking this almost always want the O(1)-space answer.

Q242 TC-038 • Basic

Find the middle of a linked list in one pass.

⚡ **Quick answer**

Use slow and fast pointers: when `fast` reaches the end, `slow` is in the middle. O(n) time, O(1) space. For even lengths this returns the second middle node.

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i], h); return h; }

static Node middle(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
    return slow;
}
System.out.println(middle(of(1, 2, 3, 4, 5)).val + " " + middle(of(1, 2, 3, 4, 5, 6)).val);
```

▶ **Output**

```
3 4
```

💡 **Pro tip**

The same fast/slow trick is the first step of "palindrome linked list" (find the middle, reverse the second half, compare) and of merge sort on linked lists.

© TechTribers. For personal use only. Do not redistribute.

Q243 TC-039

• Basic

Merge two sorted linked lists.

⚡ Quick answer

Use a dummy head and a tail pointer; repeatedly attach the smaller of the two heads, then attach the remaining list. O(m + n) time, O(1) extra space (nodes are reused).

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i],
h); return h; }
static String str(Node h) { StringBuilder sb = new StringBuilder(); for (; h != null; h = h.next)
sb.append(h.val).append(' '); return sb.toString().trim(); }

static Node merge(Node a, Node b) {
    Node dummy = new Node(0, null), tail = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { tail.next = a; a = a.next; } else { tail.next = b; b = b.next; }
        tail = tail.next;
    }
    tail.next = a != null ? a : b;
    return dummy.next;
}
System.out.println(str(merge(of(1, 3, 5, 7), of(2, 4, 6, 8, 10))));
```

▶ Output

```
1 2 3 4 5 6 7 8 10
```

💡 Pro tip

The dummy-head trick removes every special case for "the first node". Use it in any list problem that builds or deletes nodes.

Q244 TC-040

• Intermediate

Remove the N-th node from the end of a linked list in one pass.

⚡ Quick answer

Advance a `fast` pointer n steps, then move `fast` and `slow` together until `fast` hits the last node; `slow.next` is the node to delete. Use a dummy head so removing the first node needs no special case. O(L) time, O(1) space.

Java

```
static final class Node { int val; Node next; Node(int v, Node n) { val = v; next = n; } }
static Node of(int... v) { Node h = null; for (int i = v.length - 1; i >= 0; i--) h = new Node(v[i],
```


© TechTribers. For personal use only. Do not redistribute.

```
h); return h; }
static String str(Node h) { StringBuilder sb = new StringBuilder(); for (; h != null; h = h.next)
sb.append(h.val).append(' '); return sb.toString().trim(); }

static Node removeNthFromEnd(Node head, int n) {
    Node dummy = new Node(0, head), fast = dummy, slow = dummy;
    for (int i = 0; i < n; i++) fast = fast.next;
    while (fast.next != null) { fast = fast.next; slow = slow.next; }
    slow.next = slow.next.next;
    return dummy.next;
}
System.out.println(str(removeNthFromEnd(of(1, 2, 3, 4, 5), 2)) + " | " + str(removeNthFromEnd(of(1,
2), 2)) + " |" + str(removeNthFromEnd(of(1), 1)));
```

▶ Output

```
1 2 3 5 | 2 |
```

Q245 TC-041 • Intermediate

Design a stack that returns the minimum element in O(1).

⚡ Quick answer

Keep a second stack holding the minimum at each depth: push the new value onto it when it's ≤ the current minimum, and pop it when the popped value equals the minimum. All operations O(1).

Java

```
static final class MinStack {
    private final Deque<Integer> stack = new ArrayDeque<>(), mins = new ArrayDeque<>();
    void push(int x) { stack.push(x); if (mins.isEmpty() || x <= mins.peek()) mins.push(x); }
    int pop() { int x = stack.pop(); if (x == mins.peek()) mins.pop(); return x; }
    int top() { return stack.peek(); }
    int min() { return mins.peek(); }
}
MinStack s = new MinStack();
for (int x : new int[]{5, 3, 7, 3, 2}) s.push(x);
System.out.print(s.min() + " ");
s.pop(); System.out.print(s.min() + " ");
s.pop(); System.out.print(s.min() + " ");
s.pop(); s.pop(); System.out.println(s.min());
```

▶ Output

```
2 3 3 5
```

⚠ Interview trap

Pushing to the min-stack only when `x < min` (strictly) breaks with duplicate minimums: popping one `3` would lose the minimum while another `3` is still on the stack. Use `<=`.

© TechTribers. For personal use only. Do not redistribute.

🔁 **Q246** TC-042 • Intermediate

Implement a queue using two stacks.

⚡ **Quick answer**

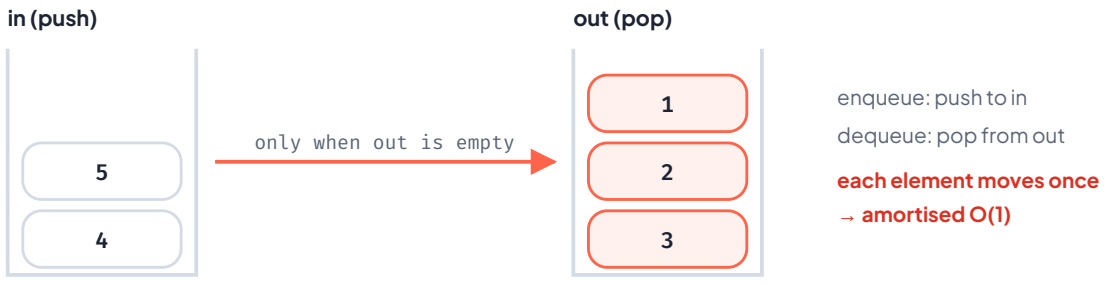
Push enqueued items onto an `in` stack. To dequeue, pop from an `out` stack; only when `out` is empty, move everything from `in` to `out` (which reverses the order). Each element moves at most once, so operations are amortised O(1).

Java

```
static final class TwoStackQueue<T> {
    private final Deque<T> in = new ArrayDeque<>(), out = new ArrayDeque<>();
    void enqueue(T x) { in.push(x); }
    T dequeue() {
        if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
        if (out.isEmpty()) throw new NoSuchElementException("queue empty");
        return out.pop();
    }
}
TwoStackQueue<Integer> q = new TwoStackQueue<>();
q.enqueue(1); q.enqueue(2); q.enqueue(3);
System.out.print(q.dequeue() + " ");
q.enqueue(4);
System.out.println(q.dequeue() + " " + q.dequeue() + " " + q.dequeue());
```

▶ **Output**

```
1 2 3 4
```



⚠️ **Interview trap**

Moving elements back from `out` to `in` after every dequeue (as some solutions do) makes each operation O(n). The transfer must happen only when `out` is empty.

© TechTribers. For personal use only. Do not redistribute.

Q247 TC-043

• Advanced

Implement an LRU cache with O(1) get and put.

⚡ Quick answer

Combine a `HashMap<key, node>` for O(1) lookup with a doubly-linked list ordered by recency. `get` moves the node to the front; `put` inserts at the front and evicts the node before the tail sentinel when over capacity.

Java

```
static final class LruCache<K, V> {
    private final class Node { K k; V v; Node prev, next; Node(K k, V v) { this.k = k; this.v = v; }
}
    private final int capacity;
    private final Map<K, Node> map = new HashMap<>();
    private final Node head = new Node(null, null), tail = new Node(null, null);
    LruCache(int capacity) { this.capacity = capacity; head.next = tail; tail.prev = head; }

    private void unlink(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }
    private void addFront(Node n) { n.next = head.next; n.prev = head; head.next.prev = n; head.next
= n; }

    V get(K k) {
        Node n = map.get(k);
        if (n == null) return null;
        unlink(n); addFront(n);
        return n.v;
    }
    void put(K k, V v) {
        Node n = map.get(k);
        if (n != null) { n.v = v; unlink(n); addFront(n); return; }
        if (map.size() == capacity) { Node lru = tail.prev; unlink(lru); map.remove(lru.k); }
        n = new Node(k, v); addFront(n); map.put(k, n);
    }
}
LruCache<Integer, String> c = new LruCache<>(2);
c.put(1, "one"); c.put(2, "two");
c.get(1);                                   // 1 becomes most recent
c.put(3, "three");                          // evicts 2
System.out.println(c.get(1) + " " + c.get(2) + " " + c.get(3));
```

▶ Output

```
one null three
```


© TechTribers. For personal use only. Do not redistribute.



💡 **Pro tip**

Interviewers usually want this hand-built version. Mention that `LinkedHashMap(cap, 0.75f, true)` with `removeEldestEntry` does the same in production, and for real caches use Caffeine (better eviction, concurrency, metrics).

Section 06

Trees, Graphs & Dynamic Programming

Recursion, BFS/DFS and the two DP problems interviewers reach for most.

Questions 248–254 · 7 questions



Q248 TC-044

• Intermediate

Implement a binary search tree with insert, search and in-order traversal.

⚡ **Quick answer**

Insert and search walk left when the key is smaller and right when larger: O(h), which is O(log n) when balanced and O(n) when skewed. In-order traversal of a BST visits keys in sorted order.

Java

```java
static final class Bst {
    private record Node(int key, Node left, Node right) { }
    private Node root;

    void insert(int key) { root = insert(root, key); }
    private Node insert(Node n, int key) {
        if (n == null) return new Node(key, null, null);
        if (key < n.key()) return new Node(n.key(), insert(n.left(), key), n.right());
        if (key > n.key()) return new Node(n.key(), n.left(), insert(n.right(), key));
        return n;                                          // ignore duplicates
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
    boolean contains(int key) {
        for (Node n = root; n != null; n = key < n.key() ? n.left() : n.right())
            if (n.key() == key) return true;
        return false;
    }
    List<Integer> inOrder() { List<Integer> out = new ArrayList<>(); walk(root, out); return out; }
    private void walk(Node n, List<Integer> out) { if (n == null) return; walk(n.left(), out);
out.add(n.key()); walk(n.right(), out); }
}
Bst t = new Bst();
for (int k : new int[]{50, 30, 70, 20, 40, 60, 80, 30}) t.insert(k);
System.out.println(t.inOrder() + " " + t.contains(60) + " " + t.contains(65));
```

▶ Output

```
[20, 30, 40, 50, 60, 70, 80] true false
```

💡 **Pro tip**

Inserting already-sorted keys produces a linked-list-shaped tree with O(n) operations. That's why `TreeMap` uses a self-balancing red-black tree.

Q249 TC-045 • Intermediate

Find the height of a binary tree and print it level by level.

⚡ **Quick answer**

Height is `1 + max(height(left), height(right))` recursively: O(n). Level-order (BFS) uses a queue and processes `queue.size()` nodes per level to group them.

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static int height(TreeNode n) { return n == null ? 0 : 1 + Math.max(height(n.left()),
height(n.right())); }

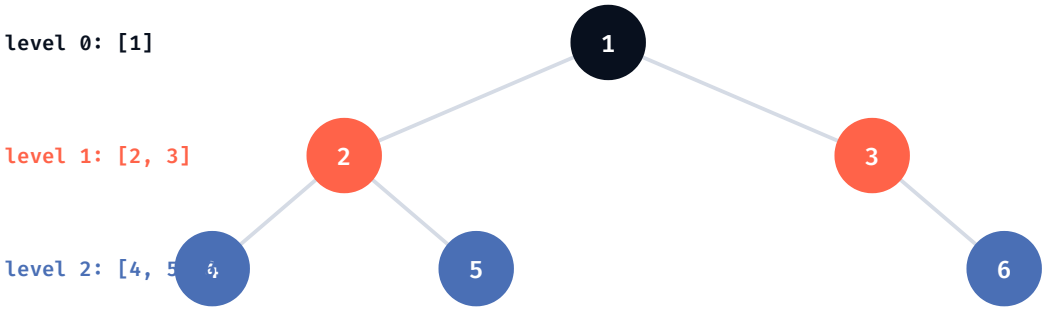
static List<List<Integer>> levels(TreeNode root) {
    List<List<Integer>> out = new ArrayList<>();
    if (root == null) return out;
    Deque<TreeNode> q = new ArrayDeque<>(List.of(root));
    while (!q.isEmpty()) {
        List<Integer> level = new ArrayList<>();
        for (int i = q.size(); i > 0; i--) {
            TreeNode n = q.poll();
            level.add(n.val());
            if (n.left() != null) q.offer(n.left());
            if (n.right() != null) q.offer(n.right());
        }
        out.add(level);
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
    return out;
}
TreeNode root = new TreeNode(1, new TreeNode(2, new TreeNode(4), new TreeNode(5)), new TreeNode(3,
null, new TreeNode(6)));
System.out.println("height=" + height(root) + " levels=" + levels(root));
```

▶ Output

```
height=3 levels=[[1], [2, 3], [4, 5, 6]]
```



Queue: poll a node, offer its children; process level by level → O(n)

✅ Q250 TC-046 • Intermediate

Check whether a binary tree is a valid binary search tree.

⚡ Quick answer

Recurse with an allowed (min, max) range: every node must be strictly inside the range set by its ancestors. O(n) time, O(h) stack. Use `long` bounds (or nullable `Integer`) so `Integer.MIN_VALUE` / `MAX_VALUE` keys work.

Java

```
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static boolean isBst(TreeNode n, long lo, long hi) {
    if (n == null) return true;
    if (n.val() <= lo || n.val() >= hi) return false;
    return isBst(n.left(), lo, n.val()) && isBst(n.right(), n.val(), hi);
}
TreeNode good = new TreeNode(5, new TreeNode(3, new TreeNode(2), new TreeNode(4)), new TreeNode(8));
TreeNode bad  = new TreeNode(5, new TreeNode(3, new TreeNode(2), new TreeNode(6)), new TreeNode(8));
// 6 > 5 on the left
System.out.println(isBst(good, Long.MIN_VALUE, Long.MAX_VALUE) + " " + isBst(bad, Long.MIN_VALUE,
Long.MAX_VALUE));
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
true false
```

⚠️ **Interview trap**

Checking only `left.val < node.val < right.val` at each node accepts the `bad` tree, because 6 is a valid right child of 3. The constraint comes from **all** ancestors, not just the parent.

Q251 TC-047 • Advanced

Find the lowest common ancestor (LCA) of two nodes in a binary tree.

⚡ **Quick answer**

Recurse: if the current node is null or one of the targets, return it. Otherwise search both subtrees; if both return non-null, the current node is the LCA, else return whichever side is non-null. O(n) time, O(h) space.

Java

```java
record TreeNode(int val, TreeNode left, TreeNode right) {
    TreeNode(int val) { this(val, null, null); }
}
static TreeNode lca(TreeNode root, int p, int q) {
    if (root == null || root.val() == p || root.val() == q) return root;
    TreeNode l = lca(root.left(), p, q), r = lca(root.right(), p, q);
    if (l != null && r != null) return root;          // p and q are on different sides
    return l != null ? l : r;
}
//          3
//        /   \
//       5     1
//      / \   / \
//     6   2 0   8
TreeNode root = new TreeNode(3,
        new TreeNode(5, new TreeNode(6), new TreeNode(2)),
        new TreeNode(1, new TreeNode(0), new TreeNode(8)));
System.out.println(lca(root, 6, 2).val() + " " + lca(root, 5, 2).val() + " " + lca(root, 6,
8).val());
```

▶ **Output**

```
5 5 3
```

🎯 **Likely follow-up**

In a BST it's simpler: walk from the root; if both values are smaller go left, if both are larger go right, otherwise you're at the LCA. O(h) with O(1) space.

© TechTribers. For personal use only. Do not redistribute.

Q252 TC-048 • Intermediate

Count the number of islands in a 2D grid.

⚡ Quick answer

Scan every cell; when you find unvisited land ('1'), increase the count and flood-fill (DFS or BFS) the whole island to mark it visited. O(rows × cols) time.

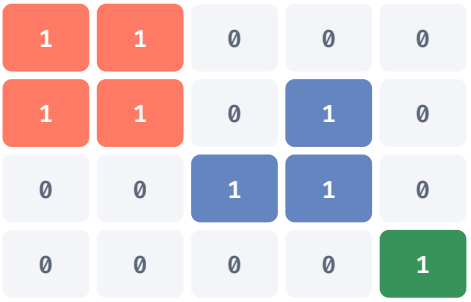
Java

```
static int islands(char[][] g) {
    int count = 0;
    for (int r = 0; r < g.length; r++)
        for (int c = 0; c < g[0].length; c++)
            if (g[r][c] == '1') { count++; sink(g, r, c); }
    return count;
}
static void sink(char[][] g, int r, int c) {
    if (r < 0 || c < 0 || r >= g.length || c >= g[0].length || g[r][c] != '1') return;
    g[r][c] = '0';
    sink(g, r + 1, c); sink(g, r - 1, c); sink(g, r, c + 1); sink(g, r, c - 1);
}
char[][] grid = {
        "11000".toCharArray(),
        "11010".toCharArray(),
        "00110".toCharArray(),
        "00001".toCharArray()};
System.out.println(islands(grid));
```

▶ Output

```
3
```

Number of islands: each DFS from an unvisited '1' sinks one whole island → 3 islands



4-directional neighbours (up/down/left/right)

mark visited by setting '1' → '0'

O(rows × cols) time

⚠ Interview trap

Recursive DFS on a huge all-land grid (e.g. 2000 × 2000) can throw `StackOverflowError`. Mention switching to an explicit stack or BFS with a queue for large inputs.

© TechTribers. For personal use only. Do not redistribute.

Q253 TC-049 • Advanced

Coin change: find the minimum number of coins to make an amount.

⚡ **Quick answer**

Bottom-up DP: `dp[a] = min over coins c of dp[a - c] + 1`, with `dp[0] = 0` and unreachable amounts kept at infinity. O(amount × coins) time, O(amount) space.

Java

```
static int minCoins(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, Integer.MAX_VALUE);
    dp[0] = 0;
    for (int a = 1; a <= amount; a++)
        for (int c : coins)
            if (c <= a && dp[a - c] != Integer.MAX_VALUE) dp[a] = Math.min(dp[a], dp[a - c] + 1);
    return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
}
static long ways(int[] coins, int amount) {                    // number of combinations
    long[] dp = new long[amount + 1];
    dp[0] = 1;
    for (int c : coins) for (int a = c; a <= amount; a++) dp[a] += dp[a - c];
    return dp[amount];
}
System.out.println(minCoins(new int[]{1, 2, 5}, 11) + " " + minCoins(new int[]{2}, 3) + " " +
minCoins(new int[]{1, 3, 4}, 6));
System.out.println(ways(new int[]{1, 2, 5}, 5));
```

▶ **Output**

```
3 -1 2
4
```

⚠️ **Interview trap**

Greedy (always take the largest coin) fails: for coins {1, 3, 4} and amount 6, greedy gives 4+1+1 = 3 coins, but 3+3 = 2 coins. Recognising when greedy breaks is the point of this question.

Q254 TC-050 • Advanced

Find the longest common subsequence (LCS) of two strings.

⚡ **Quick answer**

DP table where `dp[i][j]` is the LCS length of the first i and j characters: if the characters match, `dp[i-1][j-1] + 1`, otherwise `max(dp[i-1][j], dp[i][j-1])`. O(m × n) time and space; walk back from the corner to rebuild the subsequence.

© TechTribers. For personal use only. Do not redistribute.

LCS is the core of `diff` tools and Git's line diffs. Subsequence (characters in order, not necessarily adjacent) differs from substring (contiguous), which uses `dp[i][j] = dp[i-1][j-1] + 1` on match and **0** otherwise.

Java

```
static String lcs(String a, String b) {
    int m = a.length(), n = b.length();
    int[][] dp = new int[m + 1][n + 1];
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            dp[i][j] = a.charAt(i - 1) == b.charAt(j - 1) ? dp[i - 1][j - 1] + 1 : Math.max(dp[i -
1][j], dp[i][j - 1]);
    StringBuilder sb = new StringBuilder();
    for (int i = m, j = n; i > 0 && j > 0; ) {
        if (a.charAt(i - 1) == b.charAt(j - 1)) { sb.append(a.charAt(i - 1)); i--; j--; }
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--; else j--;
    }
    return sb.reverse().toString();
}
String r = lcs("ABCBDAB", "BDCAB");
System.out.println(r + " (length " + r.length() + ")  " + lcs("AGGTAB", "GXTXAYB"));
```

▶ Output

```
BCAB (length 4)  GTAB
```

LCS("ABCBDAB", "BDCAB") = 4: dp[i][j] = match ? diag + 1 : max(up, left)

		B	D	C	A	B
	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3
D	0	1	2	2	2	3
A	0	1	2	2	3	3
B	0	1	2	2	3	4

Time O(m × n)

Space O(m × n), or O(n) with two rolling rows

© TechTribers. For personal use only. Do not redistribute.

Series progress

254 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 6 starts at Q255.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.